

VC_SEQ

Overview

VC_SEQ contains the set of components developed to meet the needs of state patterns matching. The components include the number of state pattern detection components (vc_seq2 ... vc_seq6). State pattern detection components allow designers to match timing-constrained state patterns.

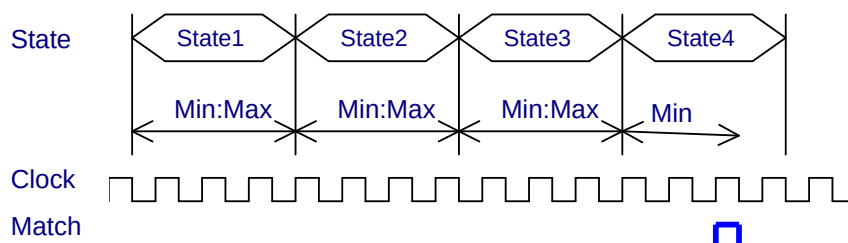
Syntax

```
vc_seq<n> #(min_time1,max_time1, min_time2, max_time2, .. min_time<n>,
severity) <Inst_name> (clk, rst_n, state1, state2, ..., state<n>, match);
```

Parameters		
Min_time<n>		Minimum limit of time state n must be valid <in the clock cycles>
Max_time<n>		Maximum limit of time state n must be valid <in the clock cycles>
Severity		Optional parameter to enable state pattern checking. When 0, no checking is done. When 1,, vc_seq treats pattern mismatch as a violation and reports an error to the Testbench
Interface		
Name	Direction	Description
clk	IN (1-bit)	System clock
rst_n	IN (1-bit)	Active-low reset
State1 ... StateN	IN (1-bit)	States defining the pattern. Each state is the one-bit signal which goes high when corresponding equation is true.
match	OUT (1-bit)	1-bit output signal, high level indicates the pattern match.

Detailed description

Vc_seq component enablen the state pattern matching in the Testbench. The following figure presents the function of the vc_seq component in more detail:



The following rules apply for the state pattern matching:

1. Minimum limit must be less or equal than the Maximum limit. Otherwise, parameter violation is reported
2. In a case Minimum limit equals 0, pattern may bypass the corresponding state. This will happen when the consequent state matching is achieved.
3. Minimum limit for the last state must be greater than 0. This will ensure the finalization of the pattern matching.

Verilog example

Current example demonstrates the usage of vc_seq component:

```
module test;

reg a, b, c, d, e, f;

reg clk, rst_n;
always #5 clk <= ~clk;

vc_seq3 #(1,1, 1,1, 1) seq1 (clk, rst_n, a_rose, b, c, match1);
vc_seq3 #(1,2, 1,2, 2) seq2 (clk, rst_n, a|b, c|d, e|f, match2);
vc_seq3 #(1,1, 1,3, 1, 1) seq3 (clk, rst_n, match1, d, e, match3);
vc_seq5 #(1,1, 2,5, 1,1, 1,1, 3, 1) seq4 (clk, rst_n, a,b,c,d,e|f|a|b, match4);

vc_seq2 #(1,1, 3) seq1 (clk, rst_n, a, b|c|d, match);
vc_seq3 #(1,1, 1,1, 3) seq1 (clk, rst_n, a, b, c|d|e, match);
vc_seq4 #(1,1, 1,1, 1,1, 4) seq1 (clk, rst_n, a, b, c, d|e|f, match);
vc_seq5 #(1,1, 1,1, 1,1, 1,1, 2) seq1 (clk, rst_n, a, b, c, d, e|f, match);

always @(posedge clk)
  if (!rst_n) begin
    a <= 1; b <= 0;
    c <= 0; d <= 0;
    e <= 0; f <= 0;
  end
  else begin
    f <= e; e <= d;
    d <= c; c <= b;
    b <= a; a <= f;
  end

initial begin
  $dumpvars;
  clk = 1;
  rst_n = 0;
  #40 rst_n = 1;
  #500 $finish;
end

endmodule
```